

Recitation 12: Implementing a Linked List Class

Topics:

- Linked lists
- Packaging a data structure as a class
- Iterators
- Copy control

Attachments

- TwoSentinels.jpg
- rec12_start.cpp
- output.txt
- Code Reading Question

Design decisions.

Our goal is to make a linked list class.

Singly or doubly linked? We will go with doubly linked. Why? It will simplify issues like how or where to add or remove a given item.

Some of our considerations will be to make it compatible with various C++ features, e.g. the ranged for and the STL (Standard Template Library) algorithms library. Providing `begin()` and `end()` methods that return iterators will buy us a lot.

However, we will also provide some functionality that the STL list class does not. After all, this is *our* class.

Sentinels: We will use *sentinels*, i.e. your class will have "dummy" nodes for the *header* (a Node before the head Node) and the *trailer* (a Node that comes after the last Node). This is to make it easier for you to write your code and is likely how you did it in your data structures course. Attached is a picture of what a List class looks with sentinels.

Nodes: The client of the `List` class will never directly interact with the `Node` type. Therefore none of the public methods will return or be passed a `Node` or `Node*`. Since no one outside of `List` will be looking at `Node`, we might as well make `Node` a nested private type.

All Nodes are, of course, on the heap. This **includes** the header and trailer Nodes which we are using for sentinel (or dummy) Nodes.

Undefined behavior: In some edge cases it may be unclear what a method should do. For example, what should a method that is supposed to return access to the first item do, e.g.. the method `front()`, if there is nothing in the list? Wherever such a question comes up, our answer will be that the behavior is *undefined*. If the behavior is undefined then you can choose to do whatever is most convenient for you. That could include the program crashing. The programmer using the method was responsible for ensuring that it is being used appropriately.

Check out often!!!

We strongly *recommend* that you check out at least a few times as you go along. Certainly, right after you finish Task One, as that is the longest task. Then perhaps after you have completed Tasks Two and Three. And after each of the subsequent tasks. Checking out a task means you know that you have done it right, and that you will at least have credit for that much! And, if everyone does this you will find less of a wait when go get checked out a last time near the end of the lab.

Draw pictures!

When a student is having trouble with a method for this lab, I will typically draw a picture of the list and then ask what needs to happen for this method to modify the list as needed. The key is looking at the picture! Perhaps identifying / numbering the steps needed! Usually the student then sees what they needed to do.

Task One

This class will naturally have a *private* struct Node. If we are constructing a doubly linked list, then the Node type will have not only data and next fields, but also a prior field. Be sure that all fields get *properly* initialized. . If you fail to initialize the pointers correctly, you will have *very* annoying bugs.

We will start by providing a simple class that supports:

- The default constructor!!!
 - It is *crucial* that you get this right to begin with, or nothing else can work! Remember you need a header node and a trailer node and they each need to point to the other. These are your *sentinels*.
- `push_back(int data)`
- `pop_back()`.
 - Note that the STL list's `pop_back` does not return anything.
- `front()`.
 - Returns the first data item in the list. (Note that's the data itself, not a Node or a pointer to the data)
 - The non-const version should be returning a reference to the data. Why? So that it can be used to *change* the data value.
 - The const version should return a constant reference to the data item, or if the

data is a primitive it could be returning a copy.

- `back()`: follows the same rules as `front()` but serves to access the last item.
- `size()`. This should, of course, be a *constant* time operation. This is one of the advantages to having the list represented by a class.
- An output operator

Modifying the list, whether adding or removing, will take a little more work than with a singly linked list. Don't be surprised if you have some segmentation faults...

Test out your functions. You should create a few lists, adding and removing items verifying that the contents are correct. (Yes, I have attached some test code. But perhaps you can come up with additional tests!)

Task Two

Now that you have a reliable list class, add the following methods and test them:

- `push_front(int data)`
- `pop_front()`
- `clear()`

Naturally it is necessary for a list to have a `clear()` method. Implement and test it. For testing, just printing may not be a sufficient test. Perhaps, after clearing first print, then add a data value and then print again. That would help catch some bugs.

Task Three

It is nice that we can *see* what's in the list, using the output operator, but we still can't *access* any of the items, other than the first and the last.

Let's solve that first with an **index operator**. With that and our size method, we can write a loop to print out the contents of a list, Write another loop to modify the value of each element and finally print out the list again.

Of course we want to be able to use the index operator on a constant List, and also use it to modify the contents of a List item.

And, of course, those loops are *horrendously* expensive! That's why the STL linked list class *does not* provide index operators.

Task Four

Ok, but wouldn't it be cool to be able to use a ranged for? We did that fairly easily when we implemented the Vector class. It will take a bit more work with a list. Why?

First, remember that we need to provide methods `begin()` and `end()`, and that their return values have to support, among other things, `operator++`. That was easy with Vector's

because we could just return the address of the first element for `begin()` and the address just past the last element for `end()`. However with the Node objects in our list, they are not laid out contiguously as the items in a `Vector` are. That has the result that incrementing the address of one Node does not take us to the next.

What can we do?

[Heads up, this is a bit of a big one...]

We will create [yet] another type, called an `iterator`.

Iterators can be really pretty simple. They will need a single field, a `Node*`. It is possible to also have them "know" what list instance they belong to, but we will leave that out for now.

In addition, iterators should support a few operations:

- `++`
 - You can stick with just a pre-increment operator, if you like.
- `--`
 - Again, you can stick with just a pre-decrement operator.
- `==`
 - Iterators will be equal if they are pointing to the same Node.
- `!=`
 - Yes, implement it.
- `*`
 - The *dereference* operator.

`iterator` also needs to provide suitable constructor(s).

Copy constructor: While discussing constructors, we should consider the copy constructor. Our `iterator` type has a pointer to an item, a Node, that exists on the heap. What does that suggest? *Not* that we need copy control. Why not? Because the iterator does not *own* the Node. So we will *not* be doing a deep copy, and in fact do not need to implement a copy constructor. The one the system provides is good enough.

We can't really test or use this class unless we have a way to get hold of an iterator for a list. The main way to do that will be through two methods in our list class `begin()` and `end()`. [You may remember that the STL's vector type supports those methods.]

The iterator that the `begin()` method returns will obviously have a pointer to the first Node. What about the iterator returned by `end()`? As we are using sentinels, it makes sense that `end` will return an iterator whose pointer points to the trailer.

What can we do with our list class now? Repeat the activities you did with the index operator, but this time using iterators. (Alright, we wrote test code for you.)

Oh, one last point (for this task, anyway), *nest* your iterator class definition inside the definition

of your List class. The name iterator is pretty common, so we could end up with naming conflicts. By nesting it, we give it a unique name, List::iterator. The user will want to be able to use the iterator type, so make sure that it is public.

Task Five

We still haven't done much to justify defining a list, let alone a list class. What's missing?

Provide an insert method for the list class. What should it be passed? The data item, certainly. And how do we know where to put the data item? Also pass in an iterator! Given an iterator for the class it should be easy enough to insert an item. But we first need to agree on where we are going to put something, given the provided iterator. Do we put it before or after?

Well, how will we insert something at the beginning of the list? The only way we have to get there is the method begin() which hands us an iterator pointing at the first item. So if we want to add something to the head of the list using these iterators, we will need to have our insert method insert *before* the iterator.

Insert should return an iterator to the new item.

One last point!!! List's insert code will need to access the Node that the iterator is pointing at. Should iterators have some method to return the Node? Remember that iterators are used by code outside the List class and that code doesn't want to / shouldn't have to know about Nodes. But the List sure does! What's the point? List methods need to be able to know what Node an iterator "points" to. How? *One* way is to have the iterator class make the List class a friend. Yes, **you can make a *whole class* a friend!** You are welcome to do it for this lab. But **don't do it** for any other assignment / lab / test question, unless we tell you to.

Task Six

Add an erase method to the class. What should it be passed? Again, an iterator! This time, there isn't any question about "where" to remove, as we will remove the Node that the iterator is pointing to.

Erase should return an iterator to the item that came after the one you erased.

Task Seven (if time allows in lab)

Copy control! Yes, we *should* be able to pass a list by value and have that result in a *copy*. We should also not have to worry about all those Node objects getting lost and creating a memory leak. And it *should* be possible to assign one list, the source, to another, the target, causing the target to be replaced by a copy of the source.

Implement the usual big three and demonstrate that they work.

While this is marked "if time allows", please note that it is definitely a useful activity that does not require much code.

Observations

We now have a fairly useful linked list class. However there are still a few things it would be nice to have. You've done enough for today, but it would be good if later you take some time to *consider* the following points:

- `const`

You may have noticed that we did not discuss `const` anywhere we have used iterators. Ideally, we want to have a notion of iterators that can be safely used on constant lists, e.g. lists that were passed by constant reference. Basic C++ notation does not give us a way to indicate whether an instance of iterator type would be able to modify the contents of the list. For that we will need an *additional type*, say, `const_iterator`, that could move over a list but not be used to change anything.

If you know how to do this, **it would be a good thing to add**, but it is not required for this assignment.,

- Our list is not templated. If you have gone over templating then how much effort will it be to turn this into a generic class?
- We have not yet experimented with the STL here. Again if you are familiar with any of the algorithms there, how well does our code work with them?

Turn in

Just one file, **rec12.cpp**

Submit

Cancel